SQL

Guess the Output of This SQL Query. 90% Get It Wrong

One query that looks simple and returns nothing. Why it happens, how to fix it, and 3 more NULL traps that catch experienced analysts.

INSIDE THIS PDF · 7 PAGES

- The query
- The answer: no rows
- The fix: use NOT EXISTS
- NULL trap 2: COUNT
- NULL trap 3: = NULL
- NULL trap 4: AVG and SUM

@thedataguy16

The query

Read the tables, then guess the output before you turn the page.

```
customers                    blocked
customer_id | name           customer_id
1           | Asha           2
2           | Ravi           NULL
3           | Neha

SELECT name
FROM customers
WHERE customer_id NOT IN (
  SELECT customer_id FROM blocked
);

Most people answer: Asha, Neha
```

Your guess?

Write it down before turning the page.

@thedataguy16

The answer: no rows

The query returns an empty result. No error, no warning. Just nothing.

Why

NOT IN (2, NULL) means: customer_id <> 2 AND customer_id <> NULL.

The NULL part

Any comparison with NULL gives UNKNOWN, not TRUE. So customer_id <> NULL is UNKNOWN for every row.

The result

TRUE AND UNKNOWN is UNKNOWN. WHERE keeps only TRUE rows, so every row is dropped.

One NULL inside a NOT IN list makes the whole filter return nothing. This happens in real reports when a lookup table has a blank row.

@thedataguy16

The fix: use NOT EXISTS

NOT EXISTS checks for a matching row instead of comparing values, so NULLs do not break it.

```
SELECT c.name
FROM customers c
WHERE NOT EXISTS (
  SELECT 1
  FROM blocked b
  WHERE b.customer_id = c.customer_id
);

Result: Asha, Neha
```

Other fix 1

Filter NULLs inside the subquery: WHERE customer_id IS NOT NULL.

Other fix 2

LEFT JOIN blocked b ... WHERE b.customer_id IS NULL.

Rule to remember

Prefer NOT EXISTS over NOT IN whenever the subquery column can be NULL.

@thedataguy16

NULL trap 2: COUNT

Same table, two counts, two different answers.

```
SELECT COUNT(*)           FROM blocked;  -- 2
SELECT COUNT(customer_id) FROM blocked;  -- 1
```

- **COUNT(*)**

  Counts rows, including rows where values are NULL.

- **COUNT(column)**

  Counts only non-NULL values in that column.

- **Where it bites**

  "How many customers gave a phone number?" Use COUNT(phone), not COUNT(*).

@thedataguy16

NULL trap 3: = NULL

This query also returns nothing, even when blank emails exist.

```
-- Wrong: always returns 0 rows
SELECT * FROM customers WHERE email = NULL;

-- Right
SELECT * FROM customers WHERE email IS NULL;
```

Why

NULL means unknown. Unknown = unknown is not TRUE, so no row matches.

Also true for <>

WHERE email <> 'a@b.com' silently skips rows where email is NULL.

@thedataguy16

NULL trap 4: AVG and SUM

Aggregates skip NULLs. That can change your answer without you noticing.

```
ratings: 5, 3, NULL, NULL

AVG(rating)                    -- 4.0  (5+3)/2
AVG(COALESCE(rating, 0))       -- 2.0  (5+3+0+0)/4
```

Which is right?

Depends on meaning. Unrated is not the same as a zero rating. Decide, then say it in your answer.

COALESCE

Replaces NULL with a value you choose. Use it deliberately, not by habit.

Interview line: "Before I filter or average, I check which columns can be NULL and how each one should be treated."

@thedataguy16